Contents

1 Introduction **1**

2 Task Breakdown **1**
2.1 Step 1: Architecture Design and System Planning 1
2.1.1 Design and its motivations 1
2.1.2 Issues 1
2.1.3 Requirements 1
2.1.4 Limitations 1
2.2 Step 2: Backend Services Implementation 2
2.2.1 Design and its motivations 2
2.2.2 Issues 2
2.2.3 Requirements 2
2.2.4 Limitations 2
2.3 Step 3: Frontend Implementation and User Experience 3
2.3.1 Design and its motivations 3
2.3.2 Issues 3
2.3.3 Requirements 3
2.3.4 Limitations 3
2.4 Step 4: Data Integration and ETL Pipeline 4
2.4.1 Design and its motivations 4
2.4.2 Issues 4
2.4.3 Requirements 4
2.4.4 Limitations 4

3 Conclusions **5**

4 Division of Work **5**

5 Technical Details **5**
5.1 Future Work 5

6 Bibliography **5**

1 Introduction

The assignment required us to design and implement a full-stack web application for browsing, managing, and tracking anime content. The app provides comprehensive features and is organized in such directories:

- **solution**
  - **Makefile**: Build automation tool providing convenient targets for managing the entire application lifecycle (start, stop, restart, status checks, dependency installation, and cleanup operations);
  - **scripts**: containing Python-based ETL (Extract, Transform, Load) scripts for populating the databases as well as Bash automation scripts;
  - **servers** structured as:
    - **backend**: Express Gateway (aggregation layer), Express MongoDB service (user data), and Spring Boot PostgreSQL service (anime catalog data);
    - **frontend**: Express UI with Handlebars templates for server-side rendering.
- **report**

2 Task Breakdown

2.1 Step 1: Architecture Design and System Planning

2.1.1 Design and its motivations

The architecture has been designed around a **microservices constellation pattern**, so to ensure a clear separation of concerns enabling independent development of different components:

- **Polyglot Technology**: Separating services allowed us to leverage the strengths of Express.js for rapid API development, SpringBoot for JPA-based data access, and MongoDB for flexible user-data storage;
- **Gateway Pattern**: The Express Gateway acts as a unified entry point, aggregating requests from the frontend and routing them to appropriate backend services;
- **Databases Persistences**: We adopted PostgreSQL for structured anime catalog data (anime, characters, people) and MongoDB for user-generated content (profiles, favorites, ratings, recommendations) that benefits from schema flexibility;

2.1.2 Issues

- **Team alignment**: Differing architectural views initially slowed decisions;
- **Design simplicity vs. feature distribution**: Clearly defining service boundaries required careful design to avoid redundancy, requiring a balance between simplicity and maintainability;
- **CORS configuration**: Correctly configuring CORS policies across frontend, gateway, and services required explicit alignment following the development needs.

2.1.3 Requirements

The architecture successfully fulfills the assignment requirements by:

- **Full-Stack Coverage**: Complete implementation from database layer to presentation layer, including data persistence, business logic, API layer, and user interface;
- **RESTful API Design**: All services expose documented REST endpoints, with Swagger/OpenAPI documentation available for developer reference;
- **Separation of Concerns**: Each service has a single, well-defined responsibility, making the codebase maintainable and testable.

2.1.4 Limitations

- The gateway acts as a central dependency, potentially limiting horizontal scalability;
- CORS policies are simplified and tailored to dev environment;
- Configuration and orchestration are static and manually managed.

2.2 Step 2: Backend Services Implementation

2.2.1 Design and its motivations

The backend implementation is structured around three independent services, each serving a distinct purpose within the microservices architecture:

- **Spring Boot PostgreSQL Service**: Implemented using Spring Data JPA, this service manages the anime catalog domain model. It exposes RESTful endpoints for anime, characters, and people entities, leveraging JPA repositories for declarative query construction and automatic CRUD operations;
- **Express MongoDB Service**: Built with Express.js and Mongoose ODM, it handles user-centric data including profiles, favorites, ratings, and recommendations. MongoDB's document-oriented model was chosen for its flexibility in managing schema evolution and its ability to handle potentially large-scale user-generated content without rigid structural constraints;
- **Express Gateway**: Acting as the API aggregation layer and single entry point for client requests, the gateway follows the `Backend for Frontend` (BFF) pattern. It consolidates multiple backend calls into unified endpoints, performs response transformation and data aggregation, and exposes a simplified, frontend-optimized API to the UI layer. This abstraction shields the frontend from backend complexity and enables independent evolution of backend services.

2.2.2 Issues

The main issues encountered during backend implementation include:

- **Error handling**: Standardizing error responses across heterogeneous services proved non-trivial. Spring Boot's default exception handling produces different response structures compared to Express middleware-based error handlers. We ensured consistent HTTP status codes, error message formats, and response structures across all services;
- **Asynchronous control flow**: Managing asynchronous operations in Express services required careful use of async/await patterns to avoid race conditions, callback hell, and unhandled promise rejections;
- **Cross-service communication**: Establishing reliable HTTP communication between the gateway and backend services required handling network failures, implementing appropriate timeout configurations, and managing connection pooling in Axios.

2.2.3 Requirements

The backend implementation satisfies the core functional requirements:

- **Dependencies and Communication**: Axios has been adopted as the primary HTTP client for inter-service communication, providing promise-based request handling and interceptor support for centralized request/response transformation;
- **API Documentation**: All services expose interactive API documentation accessible through dedicated Swagger UI endpoints. The documentation includes comprehensive request/response schemas, requirements, example payloads and error response specifications;
- **CRUD Operations**: The implementation follows RESTful conventions with appropriate HTTP methods and returns standard HTTP status codes for operation outcomes;

2.2.4 Limitations

At this stage, the backend presents the following limitations:

- **Caching**: No caching layer is implemented for frequently accessed catalog data, resulting in repeated database queries for popular anime or character information;
- **Validation scope**: Business rule validation is limited and primarily delegated to application logic rather than being enforced at the database level through constraints or triggers;

2.3 Step 3: Frontend Implementation and User Experience

2.3.1 Design and its motivations

The frontend is implemented as a server-side rendered application using Express.js with Handlebars templating engine, serving dynamic HTML pages generated on the server. This design choice was motivated by several factors:

- **Server-Side Rendering**: Provides faster initial page loads by delivering fully-rendered HTML to the browser, eliminating the need for client-side framework initialization and API calls before content appears. This approach also improves SEO capabilities, which is particularly important for content discovery applications where users may find anime through search engines;
- **Handlebars Templating**: Allows for clean separation of presentation logic from business logic through a logic-less templating approach. Partials enable component reusability across different views, promoting consistency in UI elements;
- **Responsive Design**: CSS styling with media queries and flexible layouts ensures the application adapts to different viewport sizes, providing optimal user experience across desktop monitors, tablets, and mobile devices. The layout employs a mobile-first approach with breakpoints at standard device widths, utilizing flexbox and grid layouts for responsive component arrangement.

2.3.2 Issues

The main challenges encountered during frontend development include:

- **Error handling**: Implementing comprehensive error boundaries to catch and display user-friendly error messages when backend services fail or return unexpected responses;
- **Performance optimization**: Managing large result sets efficiently, particularly when displaying extensive anime catalogs or user collections. This was addressed through pagination with configurable page sizes, loading of images to reduce initial payload, and partial rendering techniques that only update modified portions of the page;
- **Template organization**: Managing growing template complexity as features expanded required establishing clear conventions for partial usage, helper organization, and data preprocessing in route handlers to keep templates focused on presentation rather than logic.

2.3.3 Requirements

The frontend implementation fulfills the assignment requirements by providing:

- **Complete User Workflows**: All core features including anime browsing, character exploration, user profile management, favorites tracking, and rating submission are accessible through intuitive navigation structures;
- **Data Visualization**: User and animes' statistics are displayed with visual score indicators including star ratings, numerical scores, and color-coded badges that provide immediate visual feedback on content quality. Aggregate statistics show popularity metrics and ranking information;
- **CRUD Interfaces**: Comprehensive forms enable managing user favorites with add/remove functionality accessible from both anime detail pages and dedicated collection views. Character creation interface provide multi-field forms with validation feedback, while character removal includes confirmation dialogs to prevent accidental deletions. All forms implement CSRF protection and provide clear success/error feedback;
- **Search and Filtering**: Advanced search capabilities allow users to find anime by title, genre, or release year, with filter controls that combine multiple criteria.

2.3.4 Limitations

- **Client-side validation**: Form validation is primarily handled on the server side through Express middleware and backend service validation, resulting in limited immediate client-side feedback;
- **Pagination dependency**: Handling large result sets relies entirely on backend-driven pagination with server-controlled page sizes and navigation, which limits client-side interaction flexibility;
- **No offline support**: The application does not implement service workers, application caching, or offline-first strategies, making it completely dependent on network connectivity.

2.4 Step 4: Data Integration and ETL Pipeline

2.4.1 Design and its motivations

The data integration layer bridges cleaned MyAnimeList datasets from a previously developed data analysis project with the application databases.
This layer was implemented using Python scripts to perform ETL operations:

- **PostgreSQL Loading**: The `data_load_pgsql.py` script handles bulk insertion of anime catalog data (anime, characters, people, and their relationships) using psycopg2 with batch insert optimization;
- **MongoDB Loading**: The `data_load_mongo.py` script implements chunked insertions for user-related data (profiles, favorites, ratings, recommendations, statistics) using pymongo with configurable batch sizes to manage memory consumption;
- **Data Transformation**: The scripts include transformation logic to convert CSV data types to appropriate database types, handle null values, and establish proper foreign key references;
- **Idempotency**: Both scripts support clearing existing collections/tables before loading, enabling repeatable development workflows without manual database cleanup.

2.4.2 Issues

- **Large data volumes**: Managing high-volume datasets required chunked processing to control memory usage and execution time at loading time;
- **Character encoding**: Ensuring consistent UTF-8 encoding was necessary to correctly store Japanese characters and special symbols.

2.4.3 Requirements

The ETL pipeline fulfills the data integration requirements:

- **Complete Data Transfer**: All cleaned datasets from the data analysis project are successfully loaded into the application databases without data loss;
- **Data Integrity**: Referential integrity is maintained across related entities, with validation checks;
- **Error Handling**: Comprehensive error messages and logging enable troubleshooting of data quality issues or database connectivity problems.

2.4.4 Limitations

- **Full data reloads**: The pipeline replaces entire datasets rather than supporting incremental or delta-based updates;
- **Schema coupling**: Data transformations are tightly coupled to the current database schemas, reducing flexibility to schema evolution;
- **Limited validation**: Business rule validation during loading is minimal and focuses primarily on structural integrity;
- **No transactional rollback**: Failed batch operations do not trigger automated rollback mechanisms across the full dataset.

3 Conclusions

The design and implementation of this full-stack application provided valuable insights into the development of production-oriented web systems based on a microservices architecture.

- **Microservices Benefits and Complexity**: The separation of concerns across multiple services enabled parallel development and technology heterogeneity, but also introduced operational complexity;
- **Polyglot Persistence**: The combined use of PostgreSQL for structured catalog data and MongoDB for flexible user data, proved effective and efficient;
- **API Gateway Pattern**: The Express Gateway successfully abstracted backend complexity from the frontend, exposing a unified API surface. This enabled backend refactoring without impacting the UI, demonstrating the benefits of loose coupling;
- **Full-Stack Integration**: Integrating the data analysis pipeline with the web application highlighted the importance of robust ETL processes and clearly defined data transformation layers.

4 Division of Work

The project followed a **branch-based parallel development** strategy. The team collaboratively defined the initial architecture, including database selection and API contracts.
Development was then divided into focused workstreams:

- **Backend Services**: Spring Boot (PostgreSQL) and Express (MongoDB) services, including data models and APIs;
- **Gateway**: Initial mock endpoints followed by full integration with backend services;
- **Frontend**: Server-side rendered UI using Handlebars, enhanced with client-side interactions.

Regular synchronization and occasional pair programming ensured knowledge sharing and reduced development silos during development, testing and refactoring phases.

5 Technical Details

The application builds upon cleaned datasets produced within an external project of ours [1] .
The data cleaning Jupyter notebook must be executed first to generate the directory `~/datasets/cleaned/`. Subsequently, the ETL scripts can be run to populate the application databases.

5.1 Future Work

Potential extensions of the project include:

- Docker Compose-based containerization and deployment
- JWT authentication with role-based access control
- Redis caching layer for frequently accessed data

6 Bibliography

In accordance with the project guidelines, the primary reference materials were the official course resources. Additional tools were used to support development and documentation.

- **Generative AI Tools**: Used to explore pagination strategies and assist in refining technical writing and formatting across the project;
- **Course Materials**: TWEB course (University of Turin, 2025/2026), covering microservices architecture, RESTful API design and full-stack web development;
- **Data Foundation**: HCI Data Analysis project repository [1].

References

[1] Adam, Aseglio, Bertola. **Data Analysis Project**. Available at: `git@github.com:FrancoAseglio/Adam_Aseglio_Bertola_HCI.git`